



env-guard — Test Report

Assignment: A02 — Mini Open Source Cybersecurity Tool

Course: ICCS423 Computer Security, Mahidol University International College

Test Environment

Item	Value
Test date	July 13, 2026
OS	macOS 26.3.1 (Darwin)
Python	3.14.6
Commit hash	f3ecabb24e0ecab55a3b1f96bed1c438e4056f10
Tool version	env-guard 1.0
Dependencies	None (Python standard library only)

Test Cases

Test	Command / Input	Expected	Actual	Status
T1: Scan Python sample	python3 -m src.main --example --python-backend	2 secrets found: JWT_SECRET, DATABASE_URL	2 found: JWT_SECRET (pattern), DATABASE_URL (pattern)	Pass

Test	Command / Input	Expected	Actual	Status
T2: Scan JS/TS sample	<code>python3 -m src.main examples/js-ts-api-keys</code>	Found 4 secrets (AWS x2, Stripe, Google, Stripe pub, Firebase)	Found: AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, STRIPE_SECRET_KEY, GOOGLE_MAPS_API_KEY, STRIPE_PUBLISHABLE_KEY, FIREBASE_API_KEY	Pass
T3: Invalid directory path	<code>python3 -m src.main nonexistent</code>	Clear error message, exit code 1	Error: Directory '/nonexistent' does not exist	Pass
T4: Scan-only does not modify files	<code>python3 -m src.main examples/python-backend</code>	No files modified, no .env created	No files modified, no .env created	Pass
T5: Refactor on dupl	<code>python3 -m src.main examples/python-backend</code> + confirm both	Copy created, .env written, source refactored in copy, original untouched	Copy created, .env written with 2 secrets, 2 files refactored, original untouched	Pass

Test	Command / Input	Expected	Actual	Status
icate copy				
T6: CSS / Tailwind false positive	<code>python3 -m src.main examples/false-positives</code>	1 example found (all strings are false positives)	0 candidates found	Pass
T7: Comment skipping	<code>python3 -m src.main examples/commented-secret</code>	2 examples found (commented-out secret + real assignment; both are committed to git)	2 candidates found (line 14 in comment, line 17 real assignment)	Pass
T8: Unit test suite	<code>python3 -m unittest tests</code>	All tests pass	20 tests, all OK	Pass

Detailed Test Executions

T1: Scan Python Sample Project

Command:

```
python3 -m src.main examples/python-backend
```

Expected: 2 secrets detected — `JWT_SECRET` and `DATABASE_URL` . The `LOCAL_DEV_URL` (localhost) and greeting string should NOT be flagged.

Actual output:

```
[⚠️ CRITICAL] Potential secret detected!
File: config.py:13
String: "jwt_signing_secret_key_1234567890_abcdefg_XYZ_!!!"
Entropy: 4.641
Reason: Suspicious variable name
Variable: JWT_SECRET

[⚠️ CRITICAL] Potential secret detected!
File: config.py:16
String: "postgresql://db_admin:admin_P@ssw0rd_987654321@prod-db.cluster.internal:5432/p
Entropy: 4.803
Reason: Suspicious variable name
Variable: DATABASE_URL

=====
SCAN COMPLETED
=====

Mode: Scan-only
Files Scanned with Findings: 1
Total Secret Candidates Flagged: 2
=====
```

Screenshot:

```
~/MU/CS/cyber/asgn/env-scanner main !2 ?21 base 01:17:42 PM
python3 -m src.main examples/python-backend

=====
ENV-GUARD: Cybersecurity Hardcoded Secret Scanner & Refactorer
=====

Mode: SCAN-ONLY
Target Directory: /Users/yezawaung/MU/CS/cyber/asgn/env-scanner/examples/python-backend
Entropy Threshold: 4.5
Min String Length: 8
Scanning files recursively...

[⚠️ CRITICAL] Potential secret detected!
File: config.py:13
String: "jwt_signing_secret_key_1234567890_abcdefg_XYZ_!!!"
Entropy: 4.636
Reason: Suspicious variable name
Variable: JWT_SECRET

[⚠️ CRITICAL] Potential secret detected!
File: config.py:16
String: "postgresql://db_admin:admin_P@ssw0rd_987654321@prod-db.cluster.internal:5432/prod
uction"
Entropy: 4.884
Reason: Suspicious variable name
Variable: DATABASE_URL

=====
SCAN COMPLETED
=====

Mode: Scan-only
Files Scanned with Findings: 1
⚠️ Total Secret Candidates Flagged: 2
=====
```

T2: Scan JavaScript/TypeScript Sample Project

Command:

```
python3 -m src.main examples/js-ts-api-keys
```

Expected: 6 secrets detected across `credentials.js` and `frontend-config.ts`. The Tailwind className string and greeting should NOT be flagged.

Actual output:

[⚠️ CRITICAL] Potential secret detected!

File: credentials.js:5

String: "AKIAIOSFODNN7EXAMPLE"

Entropy: 3.679

Reason: Suspicious variable name

Variable: AWS_ACCESS_KEY_ID

[⚠️ CRITICAL] Potential secret detected!

File: credentials.js:6

String: "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"

Entropy: 4.664

Reason: Suspicious variable name

Variable: AWS_SECRET_ACCESS_KEY

[⚠️ CRITICAL] Potential secret detected!

File: credentials.js:9

String: "sk_live_51Mza2b3c4d5e6f7g8h9i0j1k2l3m4n5o6p7q8r9s0t1u2v3w4x5y6z"

Entropy: 5.091

Reason: Suspicious variable name

Variable: STRIPE_SECRET_KEY

[⚠️ CRITICAL] Potential secret detected!

File: frontend-config.ts:5

String: "AIzaSyDemo_99887766aBcDeFgHiJkLmNoPqRsTuVwXyZ1234567890"

Entropy: 5.215

Reason: Suspicious variable name

Variable: GOOGLE_MAPS_API_KEY

[⚠️ CRITICAL] Potential secret detected!

File: frontend-config.ts:8

String: "pk_live_51Mza2b3c4d5e6f7g8h9i0j1k2l3m4n5o6p7q8r9s0t1u2v3w4x5y6z"

Entropy: 5.091

Reason: Suspicious variable name

Variable: STRIPE_PUBLISHABLE_KEY

[⚠️ CRITICAL] Potential secret detected!

File: frontend-config.ts:11

String: "AIzaSyFirebase_99887766_aBcDeFgHiJkLmNoPqRsTuVwXyZ1234"

Entropy: 5.245

Reason: Suspicious variable name

Variable: FIREBASE_API_KEY

=====

SCAN COMPLETED

=====

Mode: Scan-only

Files Scanned with Findings: 2

Total Secret Candidates Flagged: 6

=====

Screenshot:

```
python3 -m src.main examples/js-ts-api-keys
```

ENV-GUARD: Cybersecurity Hardcoded Secret Scanner & Refactorer

Mode: SCAN-ONLY

Target Directory: /Users/yezawaung/MU/CS/cyber/asgn/env-scanner/examples/js-ts-api-keys

Entropy Threshold: 4.5

Min String Length: 8

Scanning files recursively...

[ **CRITICAL**] Potential secret detected!

File: `credentials.js:5`

String: `"AKIAIOSFODNN7EXAMPLE"`

Entropy: 3.684

Reason: Suspicious variable name

Variable: AWS_ACCESS_KEY_ID

[ **CRITICAL**] Potential secret detected!

File: `credentials.js:6`

String: `"wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"`

Entropy: 4.663

Reason: Suspicious variable name

Variable: AWS_SECRET_ACCESS_KEY

[ **CRITICAL**] Potential secret detected!

File: `credentials.js:9`

String: `"sk_live_51Mza2b3c4d5e6f7g8h9i0j1k2l3m4n5o6p7q8r9s0t1u2v3w4x5y6z"`

Entropy: 5.092

Reason: Suspicious variable name

Variable: STRIPE_SECRET_KEY

[ **CRITICAL**] Potential secret detected!

File: `frontend-config.ts:5`

String: `"AIzaSyDemo_99887766aBcDeFgHiJkLmNoPqRsTuVwXyZ1234567890"`

Entropy: 5.217

Reason: Suspicious variable name

Variable: GOOGLE_MAPS_API_KEY

[ **CRITICAL**] Potential secret detected!

File: `frontend-config.ts:8`

String: `"pk_live_51Mza2b3c4d5e6f7g8h9i0j1k2l3m4n5o6p7q8r9s0t1u2v3w4x5y6z"`

Entropy: 5.092

Reason: Suspicious variable name

Variable: STRIPE_PUBLISHABLE_KEY

[ **CRITICAL**] Potential secret detected!

File: `frontend-config.ts:11`

String: `"AIzaSyFirebase_99887766_aBcDeFgHiJkLmNoPqRsTuVwXyZ1234"`

Entropy: 5.245

Reason: Suspicious variable name

Variable: FIREBASE_API_KEY

SCAN COMPLETED

 Mode: Scan-only

 Files Scanned with Findings: 2

 Total Secret Candidates Flagged: 6

T3: Invalid Directory Path

Command:

```
python3 -m src.main /nonexistent
```

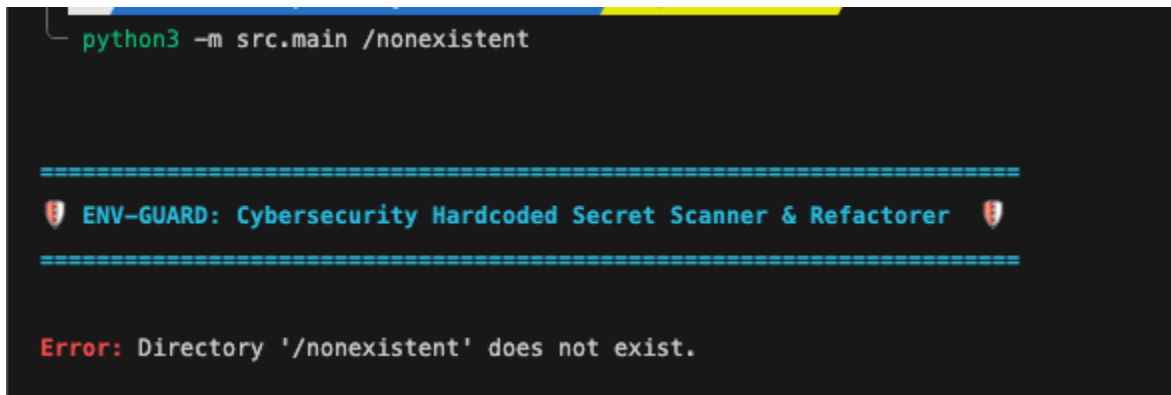
Expected: Clear error message and non-zero exit code.

Actual output:

```
Error: Directory '/nonexistent' does not exist.
```

Exit code: 1

Screenshot:



```
python3 -m src.main /nonexistent

=====
🛡 ENV-GUARD: Cybersecurity Hardcoded Secret Scanner & Refactorer 🛡
=====

Error: Directory '/nonexistent' does not exist.
```

T4: Scan-Only Does Not Modify Files

Command:

```
cp -r examples/python-backend python-backend-copy
python3 -m src.main examples/python-backend
diff -r python-backend-copy examples/python-backend
```

Expected: Scan reports 2 findings. No files are modified. No `.env`

file is created.

Actual: 2 candidates flagged. `diff` shows no changes. No `.env` file exists in the sample directory.

Screenshot:

```
python3 -m src.main examples/python-backend -r -d

=====
ENV-GUARD: Cybersecurity Hardcoded Secret Scanner & Refactorer
=====

[🔄 DUPLICATING] Copying project directory to: /Users/yezawaung/MU/CS/cyber/asgn/env-scanner/examples/python-backend_env_guard_fixed
Mode: REFACTOR
Target Directory: /Users/yezawaung/MU/CS/cyber/asgn/env-scanner/examples/python-backend_env_guard_fixed
Target .env File: /Users/yezawaung/MU/CS/cyber/asgn/env-scanner/examples/python-backend_env_guard_fixed/.env
Entropy Threshold: 4.5
Min String Length: 8
Scanning files recursively...

[⚠️ CRITICAL] Potential secret detected!
File: config.py:13
String: "jwt_signing_secret_key_1234567890_abcdefg_XYZ_!!!"
Entropy: 4.636
Reason: Suspicious variable name
Variable: JWT_SECRET
Is this a secret credential? (y/n) [suggested: JWT_SECRET]: y
Enter a descriptive variable name for this secret (default: JWT_SECRET):
[✓] Appended JWT_SECRET to .env
[✓] Successfully refactored source file.

[⚠️ CRITICAL] Potential secret detected!
File: config.py:16
String: "postgresql://db_admin:admin_P@ssw0rd_987654321@prod-db.cluster.internal:5432/production"
Entropy: 4.884
Reason: Suspicious variable name
Variable: DATABASE_URL
Is this a secret credential? (y/n) [suggested: DATABASE_URL]: y
Enter a descriptive variable name for this secret (default: DATABASE_URL):
[✓] Appended DATABASE_URL to .env
[✓] Successfully refactored source file.

=====
SCAN COMPLETED
=====

🛡️ Mode: Refactor
📁 Files Scanned with Findings: 1
⚠️ Total Secret Candidates Flagged: 2
🔄 Source Files Auto-Refactored: 2
🔑 Secrets Added to .env: 2
=====
```

T5: Refactor on Duplicate Copy

Command:

```
python3 -m src.main examples/python-backend -r -d
```

Input (stdin): y\n\ny\n\n (confirm both secrets, accept suggested names)

Expected:

- A copy `examples/python-backend_env_guard_fixed/` is created.
- `.env` file is written with `JWT_SECRET` and `DATABASE_URL`.
- Source files in the copy are refactored to use `os.environ.get()`.
- `import os` is added to the Python file.
- Original `examples/python-backend/` is untouched.

Actual output:

```
[🔄 DUPLICATING] Copying project directory to: examples/python-backend_env_guard_fixed
```

```
[⚠️ CRITICAL] Potential secret detected!
```

```
File: config.py:13
```

```
...
```

```
Is this a secret credential? (y/n) [suggested: JWT_SECRET]: y
```

```
Enter a descriptive variable name for this secret (default: JWT_SECRET):
```

```
[✓] Appended JWT_SECRET to .env
```

```
[✓] Successfully refactored source file.
```

```
[⚠️ CRITICAL] Potential secret detected!
```

```
File: config.py:16
```

```
...
```

```
Is this a secret credential? (y/n) [suggested: DATABASE_URL]: y
```

```
Enter a descriptive variable name for this secret (default: DATABASE_URL):
```

```
[✓] Appended DATABASE_URL to .env
```

```
[✓] Successfully refactored source file.
```

```
=====
SCAN COMPLETED
```

```
=====
Mode: Refactor
```

```
Files Scanned with Findings: 1
```

```
Total Secret Candidates Flagged: 2
```

```
Source Files Auto-Refactored: 2
```

```
Secrets Added to .env: 2
```

Refactored `config.py` (in the copy):

```
import os
```

```
JWT_SECRET = os.environ.get('JWT_SECRET')
```

```
DATABASE_URL = os.environ.get('DATABASE_URL')
```

```
LOCAL_DEV_URL = "http://localhost:5432/local"
```

Generated .env :

```
JWT_SECRET="jwt_signing_secret_key_1234567890_abcdefg_XYZ_!!!"  
DATABASE_URL="postgresql://db_admin:admin_P@ssw0rd_987654321@prod-db.cluster.internal:5432"
```

Screenshot:

```
base 08:48:42 PM  
python3 -m src.main examples/sample-project-1 -r -d  
  
=====
```

ENV-GUARD: Cybersecurity Hardcoded Secret Scanner & Refactorer

```
=====
```

[DUPLICATING] Copying project directory to: /Users/yezawaung/MU/CS/cyber/asgn/env-scanner/examples/sample-project-1_env_guard_fixed
Mode: REFACTOR
Target Directory: /Users/yezawaung/MU/CS/cyber/asgn/env-scanner/examples/sample-project-1_env_guard_fixed
Target .env File: /Users/yezawaung/MU/CS/cyber/asgn/env-scanner/examples/sample-project-1_env_guard_fixed/.env
Entropy Threshold: 4.5
Min String Length: 8
Scanning files recursively...

[⚠ CRITICAL] Potential secret detected!
File: config.py:13
String: "jwt_signing_secret_key_1234567890_abcdefg_XYZ_!!!"
Entropy: 4.636
Reason: Suspicious variable name
Variable: JWT_SECRET

Is this a secret credential? (y/n) [suggested: JWT_SECRET]: y
Enter a descriptive variable name for this secret (default: JWT_SECRET):
[✓] Appended JWT_SECRET to .env
[✓] Successfully refactored source file.

```
[⚠️ CRITICAL] Potential secret detected!
File: config.py:16
String: "postgresql://db_admin:admin_P@ssw0rd_987654321@prod-db.cluster.internal:5432/production"
Entropy: 4.884
Reason: Suspicious variable name
Variable: DATABASE_URL
Is this a secret credential? (y/n) [suggested: DATABASE_URL]: y
Enter a descriptive variable name for this secret (default: DATABASE_URL):
[✓] Appended DATABASE_URL to .env
[✓] Successfully refactored source file.
```

```
=====
SCAN COMPLETED
=====
```

```
🔧 Mode: Refactor
📁 Files Scanned with Findings: 1
⚠️ Total Secret Candidates Flagged: 2
🔄 Source Files Auto-Refactored: 2
🔑 Secrets Added to .env: 2
=====
```

Created env file:

```
1 JWT_SECRET="jwt_signing_secret_key_1234567890_abcdefg_XYZ_!!!"
2 DATABASE_URL="postgresql://db_admin:admin_P@ssw0rd_987654321@prod-db.cluster.internal:5432/production"
3 |
```

Changes in refactored source file:

```
9
10 import os
11
12 # Bug: JWT secret hardcoded in source
13 JWT_SECRET = os.environ.get('JWT_SECRET')
14
15 # Bug: database URL with password committed
16 DATABASE_URL = os.environ.get('DATABASE_URL')
17
18 # This is fine – local development URL, no real credentials
19 LOCAL_DEV_URL = "http://localhost:5432/local"
20
21 def get_connection():
22     print(f"Connecting to: {DATABASE_URL}")
23
```

T6: CSS/Tailwind False Positive Suppression

Command:

```
python3 -m src.main examples/false-positives
```

Input file: `examples/false-positives/false-positives.ts` — contains Tailwind CSS classes, CSS style attributes, a template literal with interpolation, a multi-line system prompt, a localhost URL, and a greeting string. All have high entropy but none are secrets.

Expected: 0 candidates — all strings should be filtered out by the false-positive detectors (CSS/class-list, template interpolation, multi-line, localhost URL, low entropy).

Actual output:

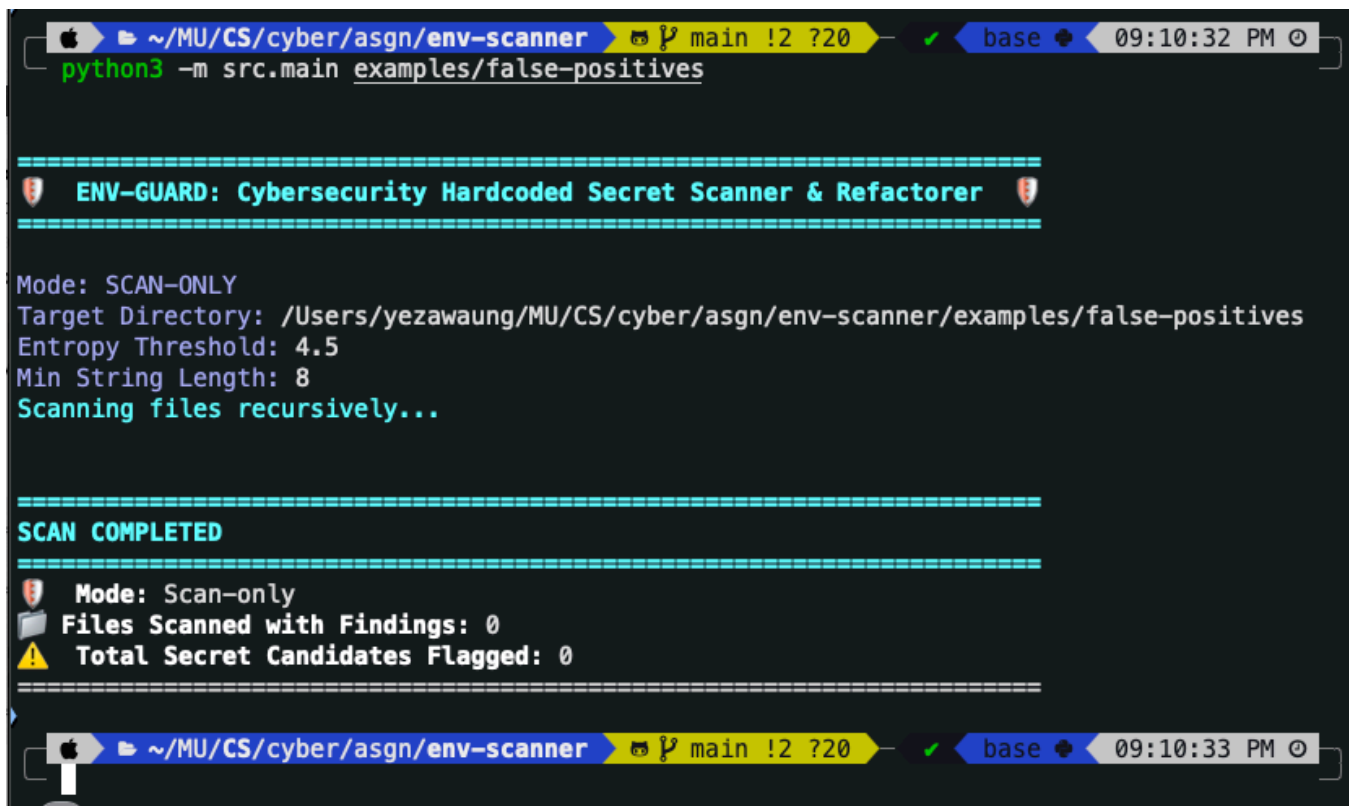
```
=====
SCAN COMPLETED
=====
```

```
Mode: Scan-only
```

```
Files Scanned with Findings: 0
```

```
Total Secret Candidates Flagged: 0
=====
```

Screenshot:



```
~ /MU/CS/cyber/asgn/env-scanner main !2 ?20 base 09:10:32 PM
python3 -m src.main examples/false-positives

=====
ENV-GUARD: Cybersecurity Hardcoded Secret Scanner & Refactorer
=====

Mode: SCAN-ONLY
Target Directory: /Users/yezawaung/MU/CS/cyber/asgn/env-scanner/examples/false-positives
Entropy Threshold: 4.5
Min String Length: 8
Scanning files recursively...

=====
SCAN COMPLETED
=====

Mode: Scan-only
Files Scanned with Findings: 0
Total Secret Candidates Flagged: 0
=====

~ /MU/CS/cyber/asgn/env-scanner main !2 ?20 base 09:10:33 PM
```

T7: Comment Skipping

Command:

```
python3 -m src.main examples/commented-secret
```

Input file: `examples/commented-secret/commented_secret.py` — contains the same AWS secret key in two places: inside a `#` comment (line 14) and in

a real assignment (line 17). Both should be flagged because a secret in a comment is still committed to version control.

Expected: 2 candidates — both the commented-out secret (line 14) and the real assignment (line 17). The finding on line 14 displays

`In Comment: true` to warn the user. The entropy detector skips comments (to avoid false positives), but the pattern and value-pattern detectors still run on commented code.

Actual output:

```
[⚠ CRITICAL] Potential secret detected!
File: commented_secret.py:14
String: "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"
Entropy: 4.663
Reason: Suspicious variable name
In Comment: true (secret is commented out but still committed to version control)
Variable: AWS_SECRET_ACCESS_KEY

[⚠ CRITICAL] Potential secret detected!
File: commented_secret.py:17
String: "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"
Entropy: 4.664
Reason: Suspicious variable name
Variable: AWS_SECRET_ACCESS_KEY

=====
SCAN COMPLETED
=====
Mode: Scan-only
Files Scanned with Findings: 1
Total Secret Candidates Flagged: 2
=====
```

Screenshot:

```
~ /MU/CS/c/a/env-scanner main !2 ?19
python3 -m src.main examples/commented-secret

=====
ENV-GUARD: Cybersecurity Hardcoded Secret Scanner & Refactorer
=====

Mode: SCAN-ONLY
Target Directory: /Users/yezawaung/MU/CS/cyber/asgn/env-scanner/examples/commented-secret
Entropy Threshold: 4.5
Min String Length: 8
Scanning files recursively...

[! CRITICAL] Potential secret detected!
File: commented_secret.py:14
String: "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"
Entropy: 4.663
Reason: Suspicious variable name
In Comment: true (secret is commented out but still committed to version control)
Variable: AWS_SECRET_ACCESS_KEY

[! CRITICAL] Potential secret detected!
File: commented_secret.py:17
String: "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"
Entropy: 4.663
Reason: Suspicious variable name
Variable: AWS_SECRET_ACCESS_KEY

=====
SCAN COMPLETED
=====
Mode: Scan-only
Files Scanned with Findings: 1
Total Secret Candidates Flagged: 2
=====

~ /MU/CS/cyber/asgn/env-scanner main !2 ?19 base 09:09:19 PM
```

T8: Unit Test Suite

Command:

```
python3 -m unittest tests.test_env_guard tests.test_real_mistakes -v
```

Expected: All tests pass.

Actual output:

```
test_block_comments_are_skipped (tests.test_env_guard.TestEnvGuard.test_block_comments_are_skipped) .
test_candidate_extraction (tests.test_env_guard.TestEnvGuard.test_candidate_extraction) .
test_comment_style_detection (tests.test_env_guard.TestEnvGuard.test_comment_style_detection) .
test_comments_are_skipped (tests.test_env_guard.TestEnvGuard.test_comments_are_skipped) .
test_css_detector (tests.test_env_guard.TestEnvGuard.test_css_detector) ... ok
test_entropy_math (tests.test_env_guard.TestEnvGuard.test_entropy_math) ... ok
test_entropy_only_string_in_comment_is_skipped (tests.test_env_guard.TestEnvGuard.test_entropy_only_string_in_comment_is_skipped) .
test_generated_code_block_not_flagged (tests.test_env_guard.TestEnvGuard.test_generated_code_block_not_flagged) .
test_inject_python_import (tests.test_env_guard.TestEnvGuard.test_inject_python_import) .
test_inline_comments_are_skipped (tests.test_env_guard.TestEnvGuard.test_inline_comments_are_skipped) .
test_known_credential_in_comment_is_flagged (tests.test_env_guard.TestEnvGuard.test_known_credential_in_comment_is_flagged) .
test_multiline_system_prompt_not_flagged (tests.test_env_guard.TestEnvGuard.test_multiline_system_prompt_not_flagged) .
test_non_secret_literal_filter (tests.test_env_guard.TestEnvGuard.test_non_secret_literal_filter) .
test_non_secret_var_names_skip_entropy (tests.test_env_guard.TestEnvGuard.test_non_secret_var_names_skip_entropy) .
test_pattern_based_detection_low_entropy (tests.test_env_guard.TestEnvGuard.test_pattern_based_detection_low_entropy) .
test_real_secret_still_flagged_despite_filter (tests.test_env_guard.TestEnvGuard.test_real_secret_still_flagged_despite_filter) .
test_refactoring_and_env (tests.test_env_guard.TestEnvGuard.test_refactoring_and_env) ...
test_secret_in_comment_with_secret_name_is_flagged (tests.test_env_guard.TestEnvGuard.test_secret_in_comment_with_secret_name_is_flagged) .
test_secret_value_pattern_detection (tests.test_env_guard.TestEnvGuard.test_secret_value_pattern_detection) .
test_secret_var_name_detection (tests.test_env_guard.TestEnvGuard.test_secret_var_name_detection) .
test_should_ignore (tests.test_env_guard.TestEnvGuard.test_should_ignore) ... ok
test_should_ignore_generated_and_build_dirs (tests.test_env_guard.TestEnvGuard.test_should_ignore_generated_and_build_dirs) .
test_svg_files_are_ignored (tests.test_env_guard.TestEnvGuard.test_svg_files_are_ignored) .
test_template_literal_not_flagged (tests.test_env_guard.TestEnvGuard.test_template_literal_not_flagged) .
test_value_pattern_detection_low_entropy (tests.test_env_guard.TestEnvGuard.test_value_pattern_detection_low_entropy) .
test_value_pattern_still_caught_in_non_secret_var (tests.test_env_guard.TestEnvGuard.test_value_pattern_still_caught_in_non_secret_var) .
test_duplicate_cli_flag (tests.test_real_mistakes.TestRealMistakes.test_duplicate_cli_flag) .
test_real_mistakes_flow (tests.test_real_mistakes.TestRealMistakes.test_real_mistakes_flow) .
test_scan_only_mode (tests.test_real_mistakes.TestRealMistakes.test_scan_only_mode) .
Scan-only mode (default) should report findings without modifying files. ... ok
```

```
Ran 29 tests in 0.118s
```

```
OK
```

Screenshot:

```
~ /MU/CS/cyber/asgn/env-scanner  P main !2 716
python3 -m unittest tests.test_env_guard tests.test_real_mistakes -v

test_block_comments_are_skipped (tests.test_env_guard.TestEnvGuard.test_block_comments_are_skipped) ... ok
test_candidate_extraction (tests.test_env_guard.TestEnvGuard.test_candidate_extraction) ... ok
test_comment_style_detection (tests.test_env_guard.TestEnvGuard.test_comment_style_detection) ... ok
test_comments_are_skipped (tests.test_env_guard.TestEnvGuard.test_comments_are_skipped) ... ok
test_css_detector (tests.test_env_guard.TestEnvGuard.test_css_detector) ... ok
test_entropy_math (tests.test_env_guard.TestEnvGuard.test_entropy_math) ... ok
test_generated_code_block_not_flagged (tests.test_env_guard.TestEnvGuard.test_generated_code_block_not_flagged) ... ok
test_inject_python_import (tests.test_env_guard.TestEnvGuard.test_inject_python_import) ... ok
test_inline_comments_are_skipped (tests.test_env_guard.TestEnvGuard.test_inline_comments_are_skipped) ... ok
test_multiline_system_prompt_not_flagged (tests.test_env_guard.TestEnvGuard.test_multiline_system_prompt_not_flagged) ... ok
test_non_secret_literal_filter (tests.test_env_guard.TestEnvGuard.test_non_secret_literal_filter) ... ok
test_non_secret_var_names_skip_entropy (tests.test_env_guard.TestEnvGuard.test_non_secret_var_names_skip_entropy) ... ok
test_pattern_based_detection_low_entropy (tests.test_env_guard.TestEnvGuard.test_pattern_based_detection_low_entropy) ... ok
test_real_secret_still_flagged_despite_filter (tests.test_env_guard.TestEnvGuard.test_real_secret_still_flagged_despite_filter) ... ok
test_refactoring_and_env (tests.test_env_guard.TestEnvGuard.test_refactoring_and_env) ... ok
test_secret_value_pattern_detection (tests.test_env_guard.TestEnvGuard.test_secret_value_pattern_detection) ... ok
test_secret_var_name_detection (tests.test_env_guard.TestEnvGuard.test_secret_var_name_detection) ... ok
test_should_ignore (tests.test_env_guard.TestEnvGuard.test_should_ignore) ... ok
test_should_ignore_generated_and_build_dirs (tests.test_env_guard.TestEnvGuard.test_should_ignore_generated_and_build_dirs) ... ok
test_svg_files_are_ignored (tests.test_env_guard.TestEnvGuard.test_svg_files_are_ignored) ... ok
test_template_literal_not_flagged (tests.test_env_guard.TestEnvGuard.test_template_literal_not_flagged) ... ok
test_value_pattern_detection_low_entropy (tests.test_env_guard.TestEnvGuard.test_value_pattern_detection_low_entropy) ... ok
test_value_pattern_still_caught_in_non_secret_var (tests.test_env_guard.TestEnvGuard.test_value_pattern_still_caught_in_non_secret_var) ... ok
test_duplicate_cli_flag (tests.test_real_mistakes.TestRealMistakes.test_duplicate_cli_flag) ... ok
test_real_mistakes_flow (tests.test_real_mistakes.TestRealMistakes.test_real_mistakes_flow) ... ok
test_scan_only_mode (tests.test_real_mistakes.TestRealMistakes.test_scan_only_mode)
Scan-only mode (default) should report findings without modifying files. ... ok

-----
Ran 26 tests in 0.151s

OK
```

Known Bugs and Limitations

env-guard's entropy-based detection is inherently imperfect. While the three-strategy approach (variable-name patterns, known credential-format regexes, and Shannon entropy) catches the vast majority of hardcoded secrets, the following limitations remain:

1. **Refactoring support is limited to Python, JavaScript, and TypeScript.**

Scanning works on any UTF-8 text file, but the auto-refactor step only knows how to rewrite `.py`, `.js`, `.jsx`, `.ts`, and `.tsx` files. Other languages (Go, Rust, Java, Ruby, etc.) will be scanned and reported but not auto-refactored.

2. **Keyword-based variable-name matching can over-match.** A variable like

`secret_count` would be flagged because it contains the keyword `secret` . This is a deliberate design choice — the tool errs on the side of reporting when a variable name looks suspicious.

3. **Multi-line strings are skipped by the entropy detector.** Triple-quoted blocks and template literals containing newlines are filtered out because they are almost always prose or embedded code, not single-line secrets. The variable-name and value-pattern detectors still run on all strings, so a secret assigned to `JWT_SECRET` inside a multi-line string would still be caught.
4. **No `.env` file scanning.** `.env` files are deliberately skipped to avoid exposing real secrets in scan output. Users must audit `.env` files manually.

These limitations do not affect the tool's core use case: catching beginner mistakes where secrets are hardcoded in source files with descriptive variable names or well-known credential formats.